

PARTICIPANT CHEAT SHEET & QUICK REFERENCE

AI-Enhanced Machine Learning & Data Workflows

for Banking & Enterprise Systems

Analytics

Pandas

Machine Learning

Generative AI

Agentic AI

LangChain

RAG

Who this is for

Software developers, data analysts & engineers, ML engineers, Python developers, banking IT & automation teams.

Pre-requisites

Basic Python programming & a basic understanding of databases / SQL queries.

How to use this

A standalone desk reference. Each section maps to a course module — skim during labs, keep after the class.

Contents

★ **Key Definitions** — glossary of every term used this week (next page)

01 Introduction to Data Analytics

02 Pandas Cheat Sheet (I) — Load & Inspect

03 Pandas Cheat Sheet (II) — Clean & Transform

04 Pandas Cheat Sheet (III) — Group & Merge

05 Exploratory Data Analysis (EDA)

06 Introduction to Machine Learning

07 ML Workflow with Scikit-learn

08 Model Evaluation & AutoML

09 Generative AI & LLM Fundamentals

10 Prompt Engineering

11 Agentic AI

12 Introduction to LangChain

13 LangChain Components

14 RAG Architecture

15 Banking & Enterprise Use Cases

16 Responsible AI + Glossary



Key Definitions — The Terms You'll Hear All Week

QUICK REFERENCE · KEEP THIS OPEN DURING THE LABS

Every term below appears repeatedly across the programme. This page is your "decoder" — one-line, plain-English meanings with a banking angle where it helps.

CORE AI & DATA

● Artificial Intelligence (AI)

Software that performs tasks needing human-like intelligence — reasoning, language, perception, decisions.

● Data Analytics

Turning raw data into insight & decisions: descriptive → diagnostic → predictive → prescriptive.

● Machine Learning (ML)

A subset of AI where models *learn patterns from data* instead of being hand-coded with rules.

● Deep Learning

ML using many-layered neural networks; powers vision, speech & the LLMs behind modern GenAI.

GENERATIVE AI

● Generative AI (GenAI)

AI that *creates* new content — text, code, images — rather than only classifying or scoring.

● Large Language Model (LLM)

A model trained on vast text that predicts the next token; the engine behind chat & copilots.

● Token & Embedding

Token: a text chunk the model reads/writes. **Embedding**: text turned into a vector of numbers capturing meaning.

● Prompt Engineering

The craft of writing clear, contextual instructions so an LLM returns reliable, useful output.

BUILDING BLOCKS — LANGCHAIN · RAG · AGENTS

● LangChain

An open-source framework providing standard building blocks to assemble LLM apps — chains, RAG, agents.

● RAG

Retrieval-Augmented Generation: fetch relevant document chunks, then have the LLM answer *grounded* in them.

● Vector Database

A store for embeddings that finds the most *similar* text fast — the memory behind RAG (FAISS, Chroma).

● Agentic AI

An LLM given a goal + tools that loops *reason* → *act* → *observe* to get work done, not just chat.

PRACTICE & DELIVERY

● Feature & Target

Feature: an input column. **Target**: the value being predicted (fraud Y/N, churn, amount).

● AutoML

Tools that auto-try many models & settings to find a strong baseline with minimal code.

● Fine-tuning

Further-training a model on your data to change its *behaviour/style* (vs RAG, which adds *knowledge*).

● Responsible AI & MLOps

Responsible AI: fair, explainable, governed use. **MLOps**: deploying, monitoring & retraining models in production.

One mental map: AI ⊃ ML ⊃ Deep Learning ⊃ Generative AI / LLMs. Analytics asks "what does the data say?", ML "what will happen?", GenAI "create/answer for me", and Agents "go do it". RAG + LangChain are *how* you wire LLMs safely to your own banking knowledge.

Data analytics is the discipline of turning raw data into **decisions**. In banking & enterprise systems it drives reporting, fraud detection, customer insight and operational automation. Analytics generally answers four escalating questions:

● **1 · Descriptive — What happened?**

Summaries of past data: totals, averages, trends.

Banking e.g. Monthly transaction volume, MIS dashboards, KPI reports.

● **2 · Diagnostic — Why did it happen?**

Drill-down, correlation, root-cause.

Banking e.g. Why did churn rise in Q3? Which segment drove it?

● **3 · Predictive — What will happen?**

ML models forecast future outcomes.

Banking e.g. Loan default probability, churn prediction, fraud scoring.

● **4 · Prescriptive — What should we do?**

Recommends actions / optimises.

Banking e.g. Best collection strategy, next-best-offer, limit decisions.

The Analytics Workflow



Key Vocabulary

Term	Meaning
Row / record	One observation (e.g. a customer, a transaction).
Column / feature	One attribute (e.g. balance, age).
Target / label	What you want to predict (e.g. fraud=Y/N).
KPI	Key Performance Indicator — a tracked metric.
ETL	Extract → Transform → Load data pipeline.
Structured	Tables/columns. <i>Unstructured</i> = text, PDFs.

The Enterprise Python Stack

Pandas	tabular data
NumPy	numeric arrays
Matplotlib / Seaborn	charts
Scikit-learn	classic ML
AutoML	automated modelling
LangChain	LLM / RAG apps
Jupyter	notebooks for exploration

AI-assisted analytics: AI coding assistants generate Pandas/SQL, explain errors and draft documentation — but you must always **validate the output against real data**.

`import pandas as pd` `import numpy as np` — the two imports nearly every script starts with. A **DataFrame** is a table; a **Series** is a single column.

Reading Data

```
# From files
df = pd.read_csv("txns.csv")
df = pd.read_excel("data.xlsx", sheet_name="Q3")
df = pd.read_json("records.json")

# Useful read_csv options
pd.read_csv("f.csv",
            sep=";", parse_dates=["date"],
            dtype={"acct":str},
            nrows=1000, na_values=["NA", "-"])

# From a database / API
df = pd.read_sql(query, conn)
df = pd.json_normalize(api_response)
```

Writing Data

```
df.to_csv("out.csv", index=False)
df.to_excel("out.xlsx", index=False)
df.to_json("out.json", orient="records")
```

First Look (always do this)

```
df.head(5)      # top rows
df.tail(5)      # bottom rows
df.shape        # (rows, cols)
df.columns      # column names
df.dtypes       # data type per col
df.info()       # types + non-null counts
df.describe()   # summary statistics
df.sample(5)    # random rows
```

Selecting Rows & Columns

Goal	Code	Returns
One column	<code>df["balance"]</code>	Series
Many columns	<code>df[["name", "balance"]]</code>	DataFrame
By label	<code>df.loc[5, "balance"]</code>	cell / slice by name
By position	<code>df.iloc[0:5, 0:3]</code>	first 5 rows, 3 cols
Filter (boolean mask)	<code>df[df["balance"] > 10000]</code>	matching rows
Multiple conditions	<code>df[(df.age>30) & (df.city=="KL")]</code>	use & ~, wrap each in ()
Membership	<code>df[df.branch.isin(["KL", "JB"])]</code>	rows in a set
Top / sort	<code>df.sort_values("amt", ascending=False).head(10)</code>	largest 10
Query syntax	<code>df.query("balance > 10000 and active")</code>	readable filter

Quick Aggregates on a Column

```
df["amt"].sum()    df["amt"].mean()
df["amt"].min()    df["amt"].max()
df["amt"].median()
df["branch"].nunique() # distinct
df["branch"].value_counts() # freq
```

Mental model: selecting is "which rows/columns do I want?"; filtering is "give me only rows where a condition is True." Almost all data prep is a chain of these two ideas.

Handling Missing Values

```
df.isna().sum()          # count NaN per col
df.dropna()              # drop rows w/ any NaN
df.dropna(subset=["amt"])
df["amt"].fillna(0)
df["amt"].fillna(df.amt.mean())
df.fillna(method="ffill") # carry forward
```

Duplicates & Types

```
df.duplicated().sum()
df.drop_duplicates(inplace=True)
df["amt"] = df.amt.astype(float)
df["date"] = pd.to_datetime(df.date)
df["id"] = df.id.astype(str)
```

Text Cleanup

```
df.name = df.name.str.strip().str.title()
df.city = df.city.str.lower()
df.code = df.code.str.replace("-", "")
mask = df.email.str.contains("@")
```

Creating & Renaming Columns

```
# new column from a formula
df["fee"] = df.amt * 0.015

# conditional column
df["tier"] = np.where(
    df.balance > 50000, "Premium", "Standard")

# map / apply
df["risk"] = df.score.apply(
    lambda s: "High" if s < 600 else "Low")
df.rename(columns={"amt": "amount"},
          inplace=True)
df.drop(columns=["temp"], inplace=True)
```

Working with Dates

```
df["year"] = df.date.dt.year
df["month"] = df.date.dt.month
df["dow"] = df.date.dt.day_name()
# filter a date range
m = (df.date >= "2025-01-01") & \
    (df.date < "2025-04-01")
q1 = df[m]
```

A Reusable Preprocessing Function (good practice)

```
def clean_transactions(df):
    df = df.drop_duplicates()
    df["date"] = pd.to_datetime(df["date"], errors="coerce")
    df["amount"] = pd.to_numeric(df["amount"], errors="coerce").fillna(0)
    df["branch"] = df["branch"].str.strip().str.upper()
    return df.dropna(subset=["date"]) # wrap your cleaning so it's repeatable & testable
```

Why wrap it in a function? Reusable preprocessing is the bridge from a notebook experiment to a production pipeline (Module 5). The same function can run on today's file, next month's file, or inside a scheduled job.

GroupBy — the engine of reporting

```
# total amount per branch
df.groupby("branch")["amount"].sum()

# multiple metrics at once
df.groupby("branch").agg(
    total=("amount", "sum"),
    avg=("amount", "mean"),
    txns=("amount", "count"))

# group by two keys
df.groupby(["branch", "month"])\
    ["amount"].sum().reset_index()
```

Pattern to remember: **split → apply → combine**. Group splits rows into buckets, the aggregation runs per bucket, results are combined into a new table.

Pivot Tables (Excel-style)

```
pd.pivot_table(df,
    index="branch",
    columns="month",
    values="amount",
    aggfunc="sum",
    fill_value=0)
```

Sorting & Ranking

```
g = df.groupby("branch")["amount"].sum()
g.sort_values(ascending=False).head(5)
df["rank"] = df.amount.rank(
    ascending=False)
```

Combining DataFrames — **merge** (SQL join)

```
full = pd.merge(txns, customers, on="customer_id", how="left") # left | right | inner | outer
combined = pd.concat([jan_df, feb_df, mar_df], ignore_index=True) # stack rows vertically
```

how=	Keeps	Typical banking use
inner	Only matching keys in both	Transactions that have a valid customer record.
left	All left rows + matches	All transactions, attach customer info where it exists.
right	All right rows + matches	All customers, even with zero transactions.
outer	Everything from both	Reconciliation — find records missing on either side.

From DataFrame to a KPI Report (end-to-end)

```
kpi = (df
    .groupby("branch")
    .agg(total=("amount", "sum"), txns=("amount", "count"), avg=("amount", "mean"))
    .sort_values("total", ascending=False)
    .reset_index())
kpi.to_excel("branch_mis_report.xlsx", index=False) # automated MIS reporting
```

Quick Plotting (Matplotlib via Pandas)

```
import matplotlib.pyplot as plt
kpi.plot(x="branch", y="total", kind="bar")
plt.title("Total Amount by Branch")
plt.tight_layout(); plt.savefig("kpi.png")
```

AI tip: Describe your DataFrame columns to an AI assistant and ask it to "write Pandas to compute monthly fee revenue by branch and export to Excel." Review the generated code, then run it on a small sample first.

Common **kind=**: **bar** **line** **hist** **box** **scatter** **pie**

EDA is the detective phase: understand the shape, quality and relationships in your data **before** building models. Skipping it is the #1 cause of misleading results.

Statistical Summaries

```
df.describe()                # numeric cols
df.describe(include="object") # text cols
df.corr(numeric_only=True)    # correlation matrix
df["churn"].value_counts(
    normalize=True)           # class balance %
```

Outlier Detection (IQR method)

```
q1, q3 = df.amt.quantile([.25, .75])
iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
outliers = df[(df.amt < lo) | (df.amt > hi)]
```

High-value outliers in transaction data are often the **fraud / anomaly** signal you are hunting for — don't blindly delete them.

Visual EDA (Seaborn)

```
import seaborn as sns
sns.histplot(df.amount)          # distribution
sns.boxplot(x="tier", y="amount",
            data=df)             # compare groups
sns.heatmap(df.corr(numeric_only=True),
            annot=True, cmap="coolwarm")
sns.scatterplot(x="income",
               y="balance", data=df)
```

Reading Correlation (−1 to +1)

Value	Meaning
≈ +1	Move together strongly
≈ 0	No linear relationship
≈ −1	Move in opposite directions

Correlation ≠ causation. A link in the data does not prove one thing causes another.

An EDA Checklist

● Shape & Types

Rows/cols, dtypes, memory. Are numbers stored as text?

● Quality

Missing %, duplicates, impossible values (negative age).

● Distributions

Histograms, skew, value ranges per feature.

● Relationships

Correlation, scatter plots, group comparisons.

● Target balance

Is the label rare (fraud 0.1%)? Affects modelling.

● Anomalies

Outliers, sudden spikes, suspicious patterns.

AI-assisted insight generation: after computing summaries, ask an LLM to "interpret these statistics for a banking audience and list 3 hypotheses to investigate." Treat its output as a starting point, then verify against the data.

Machine Learning lets a program learn patterns from historical data instead of being explicitly programmed with rules. You show it examples (features → outcome); it produces a **model** that predicts outcomes for new, unseen data.

● Supervised

Labelled data (input + known answer).

Classification: fraud / not-fraud, churn Y/N.

Regression: predict a number — credit limit, spend.

● Unsupervised

No labels — find structure.

Clustering: customer segments.

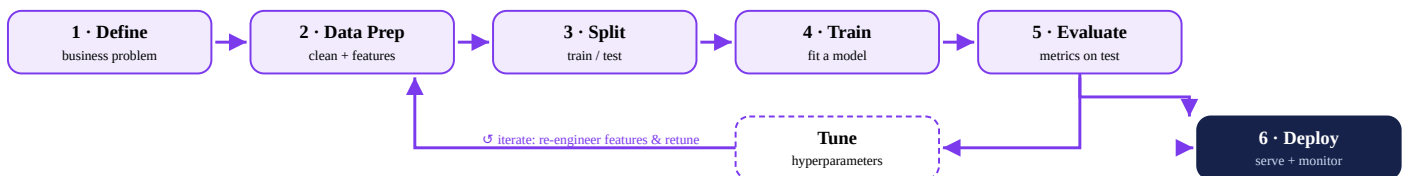
Anomaly detection: unusual transactions.

● Reinforcement

Learn by trial & reward.

Less common in core banking analytics; used in pricing, dynamic strategies, robotics.

The ML Workflow Lifecycle



Core Vocabulary

Term	Meaning
Feature (X)	Input variable used to predict.
Target (y)	The thing being predicted.
Training set	Data the model learns from.
Test set	Held-back data to check honesty.
Feature engineering	Creating better inputs from raw data.
Hyperparameter	A setting you choose (e.g. tree depth).

Overfitting vs Underfitting

Underfit — model too simple; poor on *both* train & test. *Fix:* more features, more complex model.

Good fit — strong on train, similar on test. ✓

Overfit — memorises training noise; great on train, *poor on test*. *Fix:* more data, simpler model, regularisation, cross-validation.

Golden rule: only the **test set** result counts. Never tune your model by peeking at the test data.

Which Algorithm Should I Start With?

Predict a category

Fraud / churn / approve? → **Logistic Regression** first (simple, explainable), then **Random Forest** / **XGBoost** for accuracy.

Predict a number

Spend, limit, balance? → **Linear Regression** as a baseline, then tree-based **regressors** for non-linear patterns.

Find groups / oddities

Segments or anomalies? → **K-Means** to cluster customers, **Isolation Forest** to flag unusual transactions.

Start simple, then escalate. A clear baseline you understand beats a complex model you can't explain — especially in a regulated bank. Add complexity only when the metric demands it, and always re-check on the held-out test set.

Scikit-learn gives every model the same simple interface: `.fit()` to train, `.predict()` to use. Learn it once, reuse everywhere.

Complete Classification Example — Loan Approval

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

X = df[["income", "age", "loan_amt", "credit_score"]] # features
y = df["approved"] # target (0/1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train) # 4 · TRAIN
preds = model.predict(X_test) # use on unseen data

print(classification_report(y_test, preds)) # 5 · EVALUATE
```

Common Algorithms (start simple)

Task	Try first
Classification	LogisticRegression, RandomForest
Regression	LinearRegression, RandomForestRegressor
Clustering	KMeans
Boosted (strong)	XGBoost / LightGBM

Encoding & Scaling

```
from sklearn.preprocessing import StandardScaler
# numbers on similar scale
Xs = StandardScaler().fit_transform(X)
# text categories → numbers
df = pd.get_dummies(df, columns=["city"])
```

Reusable Pipeline (Module 5)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", RandomForestClassifier())
])
pipe.fit(X_train, y_train)
pipe.predict(X_test)
```

A Pipeline bundles preprocessing + model into **one object** — so the exact same steps run in training and in production. No "I forgot to scale" bugs.

Save & Load a Model

```
import joblib
joblib.dump(pipe, "loan_model.pkl")
pipe = joblib.load("loan_model.pkl")
```

Hands-On Map (course labs → code)

Loan approval classification on customer features

Fraud detection classification + anomaly thresholds

Customer churn classification on usage/tenure

Feature engineering ratios, age buckets, recency

Explainability feature_importances_ / SHAP

Classification Metrics — know which to trust

Metric	Question it answers	Use when...
Accuracy	% of all predictions correct	Classes are balanced. Misleading for rare fraud.
Precision	Of flagged positives, how many were right?	False alarms are costly (blocking good customers).
Recall	Of real positives, how many did we catch?	Missing a case is costly (undetected fraud).
F1-score	Balance of precision & recall	You need one number for imbalanced data.
ROC-AUC	Ranking quality across thresholds	Comparing models overall.

The Confusion Matrix

		Predicted	
		Neg	Pos
Actual	Neg	TN correct neg	FP false alarm
	Pos	FN missed	TP correct pos

Precision = $TP / (TP + FP)$ • Recall = $TP / (TP + FN)$

Cross-Validation (more honest)

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y,
                          cv=5, scoring="f1")
print(scores.mean(), scores.std())
```

Explainability

```
# which features matter most?
imp = dict(zip(X.columns,
               model.feature_importances_))
# per-prediction reasons → SHAP
import shap
```

Regulated lending requires explainable decisions — a customer denied a loan may be entitled to know why. Prefer interpretable models or pair black-box models with SHAP.

AutoML — automating model selection

AutoML tools automatically try many algorithms, preprocessings and hyperparameters, then surface the best performer — compressing days of manual tuning into a configurable run.

Examples: **PyCaret** **Auto-sklearn** **H2O AutoML** **FLAML**

Cloud AutoML

AutoML	Traditional ML
Fast baseline, less code	Full control of every step
Great for prototyping	Better for bespoke needs
Can be a "black box"	Easier to reason about

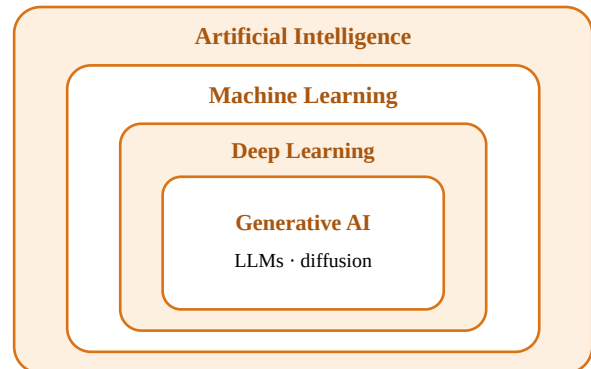
Natural-language-to-SQL & "chat with CSV" tools extend this idea: an LLM writes the query/analysis for you from a plain-English request.

Generative AI creates new content — text, code, images — rather than only classifying or predicting numbers. **Large Language Models (LLMs)** are the text engines behind it (GPT, Claude, Llama, IBM Granite, Gemini).

How an LLM Produces Text

- **Tokens:** text is split into pieces (~¾ of a word). Models read & write in tokens.
- **Prediction:** the model repeatedly predicts the *next most likely token* given everything so far.
- **Transformer + attention:** the architecture that weighs which earlier words matter for the next one.
- **Embeddings:** text mapped to vectors of numbers so similar meanings sit close together — the basis of search & RAG.
- **Temperature:** randomness dial. Low = focused/repeatable; high = creative/varied.
- **Context window:** max tokens the model can "see" at once (prompt + reply).

AI / ML / GenAI — how they nest



Open vs proprietary: proprietary APIs (GPT, Claude) are easy & powerful; open models (Llama, Granite, Mistral) can run **on-premise / private cloud** — critical when banking data cannot leave the perimeter.

What LLMs are great at — and their limits

✓ Strong at

Drafting & summarising · explaining/refactoring code · classification & extraction · Q&A over *provided* documents · transforming formats · natural-language interfaces.

✗ Weak / risky at

Exact arithmetic & precise figures · knowing events after training cut-off · staying current · guaranteeing factual accuracy.

Hallucination = confident but wrong output. LLMs predict plausible text, not verified truth. In banking, **never** trust an unverified number, policy or legal statement from an LLM. The fixes: ground answers in real data (RAG, §14), keep a human in the loop, and validate every output.

A prompt is your instruction to the model. Better prompts → better, more reliable output. The single biggest lever is **specificity + context**.

The R-T-F Pattern: Role · Task · Format

```
# Vague ✗
"Write SQL for transactions."

# Engineered ✓
"You are a senior data analyst.           ← ROLE
Write a SQL query for a table `txns`
(cols: id, acct, amount, date, branch)
that returns total amount per branch for ← TASK
March 2025, sorted high to low.
Return only the SQL, no explanation."    ← FORMAT
```

High-impact techniques

Technique	What it does
Be specific	State columns, constraints, audience.
Give examples	1–3 input → output samples (few-shot).
Step-by-step	"Think through it step by step" for reasoning.
Set format	"Return JSON with keys x, y."
Provide context	Paste the data/doc to ground the answer.
Constrain	"If unsure, say you don't know."

Developer use cases (Module 1 lab)

Generate	Python functions & SQL
Debug	"explain this error & fix"
Document	auto API / docstring drafts
Logs	"summarise these log lines, flag errors"
Refactor	"make this cleaner / faster"
Tests	"write unit tests for this function"

Anatomy of an API call

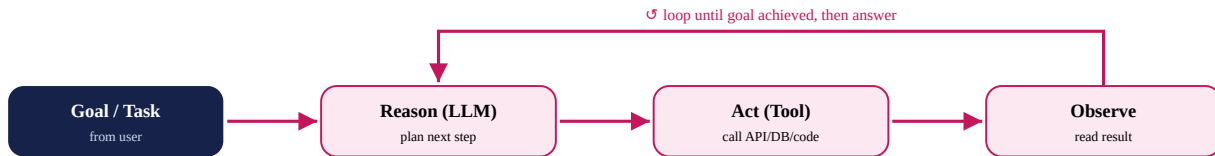
```
messages = [
  {"role": "system",
   "content": "You are a banking analyst."},
  {"role": "user",
   "content": "Summarise this report: ..."},
]
```

Roles: **system** sets behaviour/persona & rules · **user** is the request · **assistant** is the model's reply (and prior turns you send back for memory).

Security: never paste secrets, full customer PII or credentials into a public LLM. Use approved, private/enterprise endpoints for sensitive banking data, and watch for **prompt injection** in untrusted documents.

An **AI agent** is an LLM given a goal, a set of **tools**, and the ability to **loop**: reason about what to do, act, observe the result, and repeat until the goal is met. Where a chatbot only answers, an agent *does*.

The Agent Loop (Reason → Act → Observe)



Core ingredients

Part	Role
LLM brain	Reasons & decides the next action.
Tools	Functions the agent can call: search, SQL, calculator, API, code runner.
Memory	Keeps track across steps & turns.
Planner	Breaks a big goal into steps.
Orchestration	The loop/framework that runs it all.

ReAct = Reason + Act, the most common agent pattern: the model interleaves a thought, a tool action, and an observation.

Patterns & frameworks

Single agent + tools — one LLM, several tools.

Multi-agent — specialists collaborate (researcher → writer → reviewer).

Workflow / graph — defined steps with branches & checks.

Frameworks: [LangChain](#) [LangGraph](#) [CrewAI](#) [AutoGen](#)

Banking examples

Agent that reads a customer query → looks up the account (tool) → checks policy docs (RAG) → drafts a compliant reply → routes to a human for approval. Or: triage alerts, reconcile reports, summarise & file tickets.

Agents act, so guardrails matter: least-privilege tool access, approval steps for sensitive actions, spending/iteration limits, full audit logging, and a **human-in-the-loop** for anything irreversible or regulated.

LangChain is an open-source framework for building applications powered by LLMs. It provides standard building blocks — and the “glue” to connect them — so you can move from a one-off prompt to a real application: chatbots, RAG systems, agents and pipelines.

Why use it instead of raw API calls?

- **Swappable models** — change OpenAI → IBM Granite → local Llama with one line.
- **Reusable pieces** — prompts, memory, retrievers, tools as components.
- **Composition (LCEL)** — wire steps together with a simple `|` pipe.
- **RAG & agents built-in** — loaders, splitters, vector stores, agent loops ready to use.
- **Ecosystem** — integrates 100s of models, databases & tools.

Install

```
pip install langchain langchain-community
pip install langchain-openai # or langchain-ibm, etc.
```

The LangChain ecosystem

langchain-core — base abstractions & LCEL.
langchain — chains, agents, retrieval logic.
langchain-community — 3rd-party integrations.
partner packages — `langchain-openai`, `langchain-ibm`, `langchain-aws` ...
LangGraph — stateful, multi-step agent graphs.
LangSmith — tracing, debugging & evaluation.

Hello, LangChain (LCEL)

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    "Summarise this for a banker: {text}"
)
llm = ChatOpenAI(model="gpt-4o-mini")

chain = prompt | llm # compose with |
chain.invoke({"text": report})
```

LCEL = LangChain Expression Language. The pipe `|` sends the output of one component as the input to the next — exactly like a Unix pipe. `prompt | llm` | `parser` reads left-to-right.

From One Prompt to a Full Application



Each capability in this course is one more block clicked onto the last — same framework, growing power.

Component	What it is	Import / example
Chat Model	The LLM you talk to	<code>ChatOpenAI</code> , <code>ChatWatsonx</code> , <code>ChatBedrock</code>
Prompt Template	Reusable prompt with variables	<code>ChatPromptTemplate.from_template(...)</code>
Output Parser	Turns reply into clean text / JSON	<code>StrOutputParser()</code> , <code>JsonOutputParser()</code>
Chain (LCEL)	Components piped together	<code>prompt llm parser</code>
Document Loader	Reads source files	<code>PyPDFLoader</code> , <code>CSVLoader</code> , <code>WebBaseLoader</code>
Text Splitter	Chunks long docs	<code>RecursiveCharacterTextSplitter</code>
Embeddings	Text → vectors	<code>OpenAIEmbeddings</code> , <code>WatsonxEmbeddings</code>
Vector Store	Stores & searches vectors	<code>FAISS</code> , <code>Chroma</code> , <code>Milvus</code> , <code>PGVector</code>
Retriever	Fetches relevant chunks	<code>vectorstore.as_retriever()</code>
Memory	Remembers conversation	chat history / message store
Tools & Agents	Let the LLM act (§11)	<code>@tool</code> , <code>create_react_agent</code>

A chain with structured output

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | llm | StrOutputParser()
text = chain.invoke({"text": doc}) # clean string
```

Memory (multi-turn chat)

```
from langchain_core.messages import (
    HumanMessage, AIMessage)
history = [
    HumanMessage("What's my balance?"),
    AIMessage("RM 12,300."),
    HumanMessage("And last month?")]
llm.invoke(history) # context carried
```

Switching providers (on-prem / private)

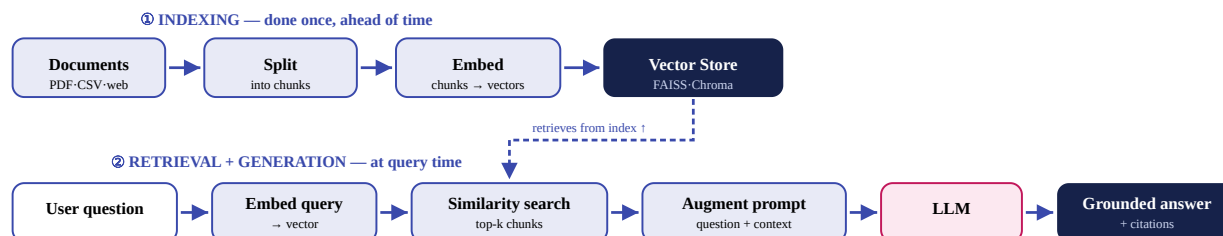
```
# IBM watsonx (e.g. Granite)
from langchain_ibm import ChatWatsonx
llm = ChatWatsonx(
    model_id="ibm/granite-3-8b-instruct",
    url=URL, project_id=PID)

# AWS Bedrock
from langchain_aws import ChatBedrock
llm = ChatBedrock(model_id="...")
```

Because every chat model shares the same interface, the rest of your chain (prompts, retrievers, parsers) stays unchanged — vital when a bank must keep data inside its own cloud.

RAG (Retrieval-Augmented Generation) connects an LLM to your own knowledge — PDFs, policies, manuals — so answers are grounded in *real* documents instead of the model's memory. This is the primary cure for hallucination and how you build a **banking knowledge assistant**.

Two phases: Indexing (offline) & Retrieval + Generation (live)



Minimal RAG with LangChain

```
docs = PyPDFLoader("policy.pdf").load()
chunks = splitter.split_documents(docs)
db = FAISS.from_documents(chunks, embeddings)
retriever = db.as_retriever(search_kwargs={"k":4})

ctx = retriever.invoke(question)
chain = rag_prompt | llm | StrOutputParser()
chain.invoke({"context":ctx, "question":question})
```

Key design choices

Choice	Guidance
Chunk size	~300–800 tokens; overlap ~10–15%.
top-k	Retrieve 3–6 chunks; too many = noise.
Embeddings	Match model to language/domain.
Citations	Return source doc + page for trust.

RAG vs fine-tuning: RAG adds *knowledge* (easy to update — just re-index). Fine-tuning changes *behaviour/style* (costlier). For "answer from our documents", choose RAG.

When RAG Gives Bad Answers — and the Fix

Irrelevant chunks retrieved

Tune chunk size/overlap, improve embeddings, add metadata filters, or re-rank results before sending to the LLM.

"I don't know" too often

Increase `k`, check the document was actually indexed, and confirm the splitter didn't cut answers in half.

Still hallucinating

Instruct: "answer only from context; if absent, say so." Show sources, and keep a human review for high-stakes replies.

Grounding instruction that works well: "Use *ONLY* the provided context to answer. Cite the source. If the context does not contain the answer, reply that you don't have that information." — this single line dramatically cuts made-up answers.

Map each course technique to a real proof-of-concept. These are the patterns behind the capstone projects.

● Fraud Detection

Approach: supervised classification + anomaly/outlier detection on transactions.

Stack: Pandas → features → Scikit-learn → recall-focused metrics.

Watch: heavy class imbalance; favour recall.

● Customer Churn

Approach: classification on tenure, usage, complaints.

Stack: EDA → feature engineering → RandomForest → SHAP for "why".

Value: target retention offers.

● Loan / Credit Decisioning

Approach: classification on income, score, history.

Must: explainable model — applicants may be owed reasons.

Output: approve / review / decline + probability.

● Document Q&A Assistant

Approach: RAG over policies, manuals, circulars.

Stack: Loader → Splitter → Embeddings → FAISS → LLM with citations.

Value: staff self-serve, fewer escalations.

● Automated MIS / KPI Reporting

Approach: Pandas groupby → pivot → Excel/chart, scheduled.

Plus: LLM writes the narrative summary of the numbers.

● Customer-Support Agent

Approach: agent (\$11) + RAG + account-lookup tool.

Flow: understand → retrieve policy → draft reply → human approves.

Capstone Blueprint — pick one, ship a POC

- 1 **Frame** the business question & success metric (e.g. "catch 80% of fraud at <5% false alarms").
- 2 **Prepare** the (synthetic) dataset — load, clean, engineer features in Pandas.
- 3 **Explore** with EDA — distributions, correlations, outliers, target balance.
- 4 **Build** the solution — ML model, or RAG pipeline, or an agent — and iterate.
- 5 **Evaluate & explain** — right metric, confusion matrix / citations, feature importance.
- 6 **Present** — a short demo + the responsible-AI risks & mitigations (next page).

Responsible AI Principles

Principle	In practice
Accuracy / Validation	Test on held-out data; verify LLM outputs against source.
Fairness	Check models don't discriminate across groups.
Explainability	Be able to justify decisions (SHAP, citations).
Privacy & security	Protect PII; keep sensitive data in approved systems.
Human-in-the-loop	People approve high-stakes / irreversible actions.
Accountability	Log, monitor & audit; assign ownership.

Governance for Banking

Align AI use with data-protection & regulatory expectations (e.g. PDPA, central-bank technology-risk guidance). Maintain model documentation, data lineage, access controls, and a review process before production.

Glossary — fast reference

LLM — large language model.

GenAI — generative AI.

Token — text unit a model reads/writes.

Embedding — text as a numeric vector.

Context window — max tokens at once.

Temperature — output randomness.

RAG — retrieval-augmented generation.

Vector store — DB for embeddings.

Chunking — splitting docs for retrieval.

Agent — LLM that uses tools in a loop.

ReAct — reason+act agent pattern.

Fine-tuning — retraining on your data.

Feature — model input variable.

Target — value being predicted.

Overfitting — memorising train data.

Precision/Recall — quality of positives caught.

AutoML — automated model search.

Hallucination — confident wrong output.

Top Risks & Mitigations

Hallucination → ground with RAG + cite + verify.

Data leakage → private endpoints, redact PII.

Prompt injection → sanitise untrusted text, limit tool power.

Bias → audit data & outcomes by segment.

Model drift → monitor performance, retrain.

Over-trust → keep humans accountable for decisions.

Remember: the model assists, the human decides. AI output is a *draft* to be checked — never an unquestioned source of truth in a regulated bank.

Secure-adoption checklist

- Approved tools & data boundaries defined
- No secrets / raw PII in prompts
- Outputs validated before action
- Logging, monitoring & rollback in place

You've reached the end — keep this as your desk reference.

Timmins Training Consulting Sdn. Bhd. · raj@consult-timmins.com · +60 11-1667 4727